fn make_randomized_response

Michael Shoemate (@Shoeboxam)

April 22, 2026

Proves soundness of `make_randomized_response` in `mod.rs` at commit f5bb719 (outdated[1]), a constructor taking a category set `categories` and probability `prob`. The mechanism returned by `make_randomized_response` takes in a data set `arg` (a single category), and...

- ...if `arg` is in `categories`, the mechanism truthfully returns the same value `arg` with probability `prob`, otherwise it lies by selecting one of the other categories uniformly at random.
- ...if `arg` is not in `categories`, it returns a category chosen uniformly at random.

1 Hoare Triple

Precondition

Compiler-verified

- Variable `categories` must be a set with members of type `T`
- Variable `prob` must be of type `f64`

Caller-verified

None

Pseudocode

```python
def make_randomized_response(categories: set[T], prob: f64):
    input_domain = AtomDomain(T)
    input_metric = DiscreteMetric()
    output_measure = MaxDivergence()

    categories = list(categories)

    if len(categories) < 2:  #
        raise ValueError("expected at least two categories")

    num_categories = len(categories)

    if not (1 / num_categories <= prob <= 1):  #
        raise ValueError("probability must be within [1/num_categories, 1]")

    # prepare constant:
    if prob == 1.0:
        c = float("inf")
    else:
```

[1] See new changes with `git diff f5bb719..6a49105ff rust/src/measurements/randomized_response/mod.rs`

```
        c = prob.inf_div((1).neg_inf_sub(prob)) \
            .inf_mul(num_categories.inf_sub(1)) \
            .inf_ln()

    def privacy_map(d_in: u32) -> QO:
        if d_in == 0:
            return 0
        else:
            return c

    def function(truth: T) -> T:  #
        index = find_index(categories, truth)
        sample = sample_uniform_uint_below(
            len(categories) - (1 if index is not None else 0))

        if index is not None and sample >= index:
            sample += 1

        lie = categories[sample]

        be_honest = sample_bernoulli_float(prob, False)
        is_member = index is not None
        return truth if be_honest and is_member else lie

    return Measurement(input_domain, function, input_metric, output_measure, privacy_map)
```

Postcondition

Theorem 1.1. For every setting of the input parameters `(categories, prob, T)` to `make_randomized_response` such that the given preconditions hold, `make_randomized_response` raises an error (at compile time or run time) or returns a valid measurement. A valid measurement has the following properties:

1. (Data-independent runtime errors). For every pair of members x and x' in `input_domain`, `invoke`(x) and `invoke`(x') either both return the same error or neither return an error.

2. (Privacy guarantee). For every pair of members x and x' in `input_domain` and for every pair (`d_in`, `d_out`), where `d_in` has the associated type for `input_metric` and `d_out` has the associated type for `output_measure`, if x, x' are `d_in`-close under `input_metric`, `privacy_map`(`d_in`) does not raise an error, and `privacy_map`(`d_in`) $=$ `d_out`, then `function`(x), `function`(x') are `d_out`-close under `output_measure`.

Proof.

> The proof assumes that `sample_uniform_uint_below` and `sample_bernoulli_float` satisfy their postconditions.

`sample_uniform_uint_below` and `sample_bernoulli_float` can only fail due to lack of system entropy. This is usually related to the computer's physical environment and not the dataset. The rest of this proof is conditioned on the assumption that these functions do not raise an exception.

Let x and x' be datasets that are `d_in`-close with respect to `input_metric`. Here, the metric is `DiscreteMetric` which enforces that `d_in` ≥ 1 if $x \neq x'$ and `d_in` $= 0$ if $x = x'$. If $x = x'$, then the output distributions on x and x' are identical, and therefore the max-divergence is 0.

Now consider the case where $x \neq x'$. For shorthand, we let p represent `prob`, the probability of returning the input, and t denote the number of categories. The implementation first converts the input set `categories` into a vector. Because the input data type is a set, each category appears exactly once, so this conversion only fixes an arbitrary order and does not change membership or multiplicity.

t must be at least two, by pseudocode line 8, as any fewer would not be useful. p is restricted to $[1/t, 1.0]$ by pseudocode line 13, as any less would not be useful.

If $p = 1$, then pseudocode line 16 sets the privacy constant to ∞. This matches the implementation. In this case, whenever x and x' are two distinct members of `categories`, the mechanism returns x and x' deterministically, so the max-divergence is ∞. Therefore the privacy bound holds when $p = 1$. For the remainder of the proof, assume $p < 1$.

We now characterize the output distribution of the mechanism. If $x \in$ `categories`, then pseudocode line 30 sets `index` to the unique position of x in the vectorized category list. The call to `sample_uniform_uint_below` samples uniformly from $\{0, \ldots, t-2\}$. If the sampled index is greater than or equal to `index`, the implementation increments it by one, so the final index is uniform over the $t-1$ positions corresponding exactly to `categories` $\setminus\{x\}$. Therefore `lie` is uniformly distributed over the categories other than x. The call to `sample_bernoulli_float(prob, False)` returns `true` with probability p, so the mechanism returns x with probability p and otherwise returns a uniformly sampled lie.

If $x \notin$ `categories`, then `index` is `None`, so `sample_uniform_uint_below` samples uniformly from all t category indices and the shift is skipped. In this branch, `is_member` is `false`, so the mechanism always returns `lie`. Therefore the output is uniformly distributed over `categories`.

For any outcome $y \in$ `categories`, the probability of observing y is one of three values:

1. If $x \in$ `categories` and $y = x$:
$$\Pr[M(x) = y \mid y = x] = p$$

2. If $x \in$ `categories` and $y \neq x$:
$$\Pr[M(x) = y \mid y \neq x \wedge x \in \texttt{categories}] = \frac{1-p}{t-1}$$

3. If $x \notin$ `categories`:
$$\Pr[M(x) = y \mid x \notin \texttt{categories}] = \frac{1}{t}$$

Claim. The probability of case 3 is bounded by cases one and two:

$$\frac{1-p}{t-1} \leq \frac{1}{t} \leq p \tag{1}$$

The upper bound follows from pseudocode line 13, which gives $p \geq 1/t$. Reusing 13,

$$\frac{1-p}{t-1} \leq \frac{1-1/t}{t-1} = \frac{1}{t}.$$

Therefore $1/t$ is also bounded below by case two ($\frac{1-p}{t-1}$).

By (1), every point probability lies between $\frac{1-p}{t-1}$ and p. Therefore, for any neighboring x, x' and any $y \in$ `categories`,

$$\frac{\Pr[M(x) = y]}{\Pr[M(x') = y]} \leq \frac{p}{(1-p)/(t-1)} = \frac{p \cdot (t-1)}{1-p}.$$

We now consider the max-divergence of the mechanism over all choices of neighboring datasets.

$$
\begin{aligned}
&\max_{x \sim x'} D_\infty(M(x), M(x')) && (2)\\
=&\max_{x \sim x'} \max_{S \subseteq Supp(M(x))} \ln\left(\frac{\Pr[M(x) \in S]}{\Pr[M(x') \in S]}\right) && (3)\\
\leq&\max_{x \sim x'} \max_{y \in Supp(M(x))} \ln\left(\frac{\Pr[M(x) = y]}{\Pr[M(x') = y]}\right) \qquad \text{Lemma 3.3 [1]} && (4)\\
\leq&\ln\left(\frac{p \cdot (t-1)}{1-p}\right) && (5)
\end{aligned}
$$

Pseudocode line 16 implements this bound with conservative rounding towards positive infinity. When `d_in` > 0 and no exception is raised in computing `c` $=$ `privacy_map`(`d_in`), then

$$\ln\left(\frac{p \cdot (t-1)}{1-p}\right) \leq \texttt{c}.$$

Therefore it has been shown that for every pair of elements $x, x' \in$ `input_domain` and every $d_{DM}(x, x') \leq$ `d_in` with `d_in` ≥ 0, if x, x' are `d_in`-close then `function`(x), `function`(x') are `privacy_map`(`d_in`)-close under `output_measure` (the Max-Divergence). □

References

[1] Shiva P. Kasiviswanathan and Adam Smith. On the "semantics" of differential privacy: A bayesian formulation. *Journal of Privacy and Confidentiality*, 6(1), June 2014.